

1.USDT余额存储

代码: `mapping(address => uint) public balances;`

USDT使用地址->数量的映射结构存储余额, 每一个区块链地址对应一个代币数值。合约所有转账操作的本质都是修改该映射里两个地址的数值, 以此实现代币资产的链上记账。

2.函数区别

`transfer(address _to, uint _value)`

```
function transfer(address _to, uint _value) public
onlyPayloadSize(2 * 32) {
    uint fee = (_value.mul(basisPointsRate)).div(10000);
    if (fee > maximumFee) {
        fee = maximumFee;
    }
    uint sendAmount = _value.sub(fee);
    balances[msg.sender] = balances[msg.sender].sub(_value);
    balances[_to] = balances[_to].add(sendAmount);
    if (fee > 0) {
        balances[owner] = balances[owner].add(fee);
        Transfer(msg.sender, owner, fee);
    }
    Transfer(msg.sender, _to, sendAmount);
}
```

这是USDT转账的核心逻辑层, 负责处理代币余额的增减和手续费计算。

- 1.安全校验: 通过`onlyPayloadSize` 防止短地址攻击。
- 2.金额拆分: 将用户的转账金额拆分为“给接收者的净金额”和“给管理员的手续费”两部分。
- 3.余额更新: 一次性扣减发送方的全额, 再分别增加接收方和管理员的余额。
- 4.时间记录: 为每一笔资金变动(包括手续费)都生成链上可追溯的事件日志。

```
function transfer(address _to, uint _value) public whenNotPaused {
    require(!isBlackListed[msg.sender]);
    if (deprecated) {
        return
        UpgradedStandardToken(upgradedAddress).transferByLegacy(msg.sender
, _to, _value);
    } else {
        return super.transfer(_to, _value);
    }
}
```

这是USDT转账的对外入口层, 它不直接处理余额, 而是做安全校验和逻辑分发。

- 1.紧急风控: `whenNotPaused`确保合约在管理员触发暂停后, 所以转账会被冻结。
- 2.合规控制: `isBlackListed`校验防止被拉黑的地址进行转账操作。
- 3.版本兼容: 支持将请求转发到升级后的新合约, 确保旧合约的接口在弃用后依然可用。
- 4.调用底层逻辑: 通过`super.transfer`, 最终执行`BasicToken`里的核心转账逻辑。

approve(address spender, uint value)函数

```
function approve(address _spender, uint _value) public
onlyPayloadSize(2 * 32) {

    // To change the approve amount you first have to reduce
the addresses`
    // allowance to zero by calling `approve(_spender, 0)` if
it is not
    // already 0 to mitigate the race condition described
here:
    // https://github.com/ethereum/EIPs/issues/
20#issuecomment-263524729
    require(!((_value != 0) && (allowed[msg.sender]
[_spender] != 0)));

    allowed[msg.sender][_spender] = _value;
    Approval(msg.sender, _spender, _value);
}
```

这是授权的核心逻辑层，负责更新授权关系。

- 1.安全前置校验：强制用户先清0再授权，防范竞态条件攻击。
- 2.更新授权映射：修改allowed二维映射，记录用户对第三方的授权额度。
- 3.记录链上事件：通过Approval事件，让授权行为公开可查。

```
function approve(address _spender, uint _value) public
onlyPayloadSize(2 * 32) {
    if (deprecated) {
        return
UpgradedStandardToken(upgradedAddress).approveByLegacy(msg.sender,
_spender, _value);
    } else {
        return super.approve(_spender, _value);
    }
}
```

这是授权的对外入口层，负责逻辑分发。

若合约已弃用，将请求转发给新版本合约的approveByLegacy方法。

如果合约未弃用：调用super.approve，即执行父合约StandardToken中定义的approve函数

transferFrom(address from, address to, uint value)函数

```
function transferFrom(address _from, address _to, uint _value)
public onlyPayloadSize(3 * 32) {
    var _allowance = allowed[_from][msg.sender];

    // Check is not needed because sub(_allowance, _value)
will already throw if this condition is not met
    // if (_value > _allowance) throw;

    uint fee = (_value.mul(basisPointsRate)).div(10000);
    if (fee > maximumFee) {
        fee = maximumFee;
    }
    if (_allowance < MAX_UINT) {
        allowed[_from][msg.sender] = _allowance.sub(_value);
    }
    uint sendAmount = _value.sub(fee);
    balances[_from] = balances[_from].sub(_value);
    balances[_to] = balances[_to].add(sendAmount);
    if (fee > 0) {
        balances[owner] = balances[owner].add(fee);
        Transfer(_from, owner, fee);
    }
    Transfer(_from, _to, sendAmount);
}
```

核心逻辑层：

- 1.以allowed授权表为权限基础，无授权则直接失败。
- 2.区分普通授权与无限授权，差异化处理额度扣减。
- 3.余额变更、手续费规则与普通transfer保持统一。
- 4.所有资金变动均触发链上事件，保证交易可追溯。

```
function transferFrom(address _from, address _to, uint _value)
public whenNotPaused {
    require(!isBlackListed[_from]);
    if (deprecated) {
        return
UpgradedStandardToken(upgradedAddress).transferFromByLegacy(msg.se
nder, _from, _to, _value);
    } else {
        return super.transferFrom(_from, _to, _value);
    }
}
```

对外入口：不处理余额、授权计算，只做前置安全拦截+路由分发。

5. 为什么用户在让另一个合约转走自己的 USDT 前，需要先调用approve?

1. 资产隔离原则

USDT的余额存储在mapping(address => uint) balances映射中，只有地址自身能直接操作自己的余额，外部合约没有默认权限动用某一地址的资产，直接修改他人balances是被合约禁止的。

2. approve的授权作用

approve会修改全局allowed[资产所有者][被授权合约] = 额度，在链上公开登记授权关系，明确允许目标合约最多可以划转自己多少数量的 USDT。

3. transferFrom的权限校验依赖

外部合约代转时要调用transferFrom，该函数第一步就会读取allowed授权表，验证自己获得了转出方的授权；如果没有提前approve，查询到的授权额度为 0，后续额度扣减的 SafeMath 减法会触发报错，交易直接回滚，无法完成转账。

4. 安全风控约束

USDT 的黑名单、暂停机制仅管控转出方状态，不会放开资产操作权限，必须依靠approve完成用户主动授权，才能让第三方合约获得合规的代转资格，避免资产被随意挪用。

6. USDT的小数位数是多少？这和普通整数金额有什么关系？

decimals固定设置为6位。

链上原始整数数值 = 日常可读金额 (USDT) $\times 10^6$ 。